

BSDS-Notes-Algebraic Computations

Krishnendu Mukhopadhyaya

October 2025

Problem Statement

Given two non-negative integers a and b (not both zero), the objective is to compute

$$\gcd(a, b) = \max\{d \in \mathbb{Z}^+ : d \mid a \text{ and } d \mid b\}.$$

That is, the largest positive integer that divides both a and b .

Key Idea

The algorithm is based on the **Division Lemma**:

For any integers a and b with $b > 0$, there exist unique integers q and r such that

$$a = bq + r, \quad 0 \leq r < b.$$

The crucial property is:

$$\gcd(a, b) = \gcd(b, r),$$

since any integer dividing both a and b also divides $r = a - bq$.

Hence, we can replace the pair (a, b) with (b, r) and repeat the process until $r = 0$.

Algorithm Description

Euclid's Algorithm: Given integers a and b , repeatedly replace the larger number by its remainder when divided by the smaller one. When one of the numbers becomes zero, the other number is the $\gcd(a, b)$.

Algorithm 1 Euclid_GCD(a, b)

Require: Integers $a \geq 0, b \geq 0$, not both zero

Ensure: $\text{gcd}(a, b)$

```
1: while  $b \neq 0$  do  
2:    $r \leftarrow a \bmod b$   
3:    $a \leftarrow b$   
4:    $b \leftarrow r$   
5: end while  
6: return  $a$ 
```

Pseudocode

Proof of Correctness

We will prove that when the algorithm terminates, it returns $\text{gcd}(a, b)$.

Invariant: At the start of each iteration, the pair (a, b) satisfies

$$\text{gcd}(a, b) = \text{gcd}(a_0, b_0),$$

where (a_0, b_0) are the original inputs.

Proof of invariant preservation:

Suppose in one iteration we have $a = bq + r$. Then, for any integer d ,

$$d \mid a \text{ and } d \mid b \iff d \mid b \text{ and } d \mid r.$$

Hence,

$$\text{gcd}(a, b) = \text{gcd}(b, r).$$

The algorithm replaces (a, b) with (b, r) , thus preserving the invariant.

Termination: Since $r < b$ and $r \geq 0$, each iteration reduces the second argument strictly. Eventually, b becomes 0, and the loop terminates.

At termination: When $b = 0$, the algorithm returns a . By the invariant, $\text{gcd}(a, b) = \text{gcd}(a_0, b_0)$ and $\text{gcd}(a, 0) = a$. Therefore, the algorithm returns the correct result.

□

Complexity Analysis

Let $T(a, b)$ denote the number of division steps in Euclid's algorithm.

- Each iteration replaces (a, b) with $(b, a \bmod b)$.
- The values decrease rapidly; in the worst case, they follow the Fibonacci sequence.

Time Complexity: If $a > b$, then the number of iterations is $O(\log b)$. Specifically, $T(a, b) \leq 5 \log_{10} b$ (Knuth, 1997).

Time complexity: $O(\log \min(a, b))$

Space Complexity: Only a few integer variables are used.

Space complexity: $O(1)$

Example

$\text{gcd}(252, 105) :$

$$252 = 105 \times 2 + 42, \quad 105 = 42 \times 2 + 21, \quad 42 = 21 \times 2 + 0$$

Hence, $\text{gcd}(252, 105) = 21$.

Polynomial Evaluation Algorithms

Problem Statement

Given a polynomial of degree n ,

$$P(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n,$$

and a value x_0 , we wish to compute $P(x_0)$ efficiently.

Naïve Evaluation (Without Preprocessing)

The straightforward approach computes each power of x_0 independently.

Analysis:

- Computing each x_0^i from scratch requires i multiplications.
- Total number of multiplications $= 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$.

Time complexity: $O(n^2)$, Space complexity: $O(1)$.

Algorithm 2 Naive.Polynomial_Eval($a_0, a_1, \dots, a_n, x_0$)

Require: Coefficients $a_0, a_1, \dots, a_n$ and value x_0 **Ensure:** $P(x_0)$

```
1:  $p \leftarrow 0$ 
2: for  $i = 0$  to  $n$  do
3:    $term \leftarrow a_i \times x_0^i$ 
4:    $p \leftarrow p + term$ 
5: end for
6: return  $p$ 
```

Observation: This method is inefficient for large n , since each power is recomputed.

Efficient Evaluation (With Preprocessing) — Horner's Rule

Horner's rule rewrites the polynomial as:

$$P(x) = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1} + xa_n) \dots)).$$

This nested form allows evaluation using a single loop with only n multiplications and n additions.

Algorithm 3 Horner.Polynomial_Eval($a_0, a_1, \dots, a_n, x_0$)

Require: Coefficients $a_0, a_1, \dots, a_n$ and value x_0 **Ensure:** $P(x_0)$

```
1:  $p \leftarrow a_n$ 
2: for  $i = n - 1$  downto  $0$  do
3:    $p \leftarrow a_i + x_0 \times p$ 
4: end for
5: return  $p$ 
```

Analysis:

- Each iteration performs one multiplication and one addition.
- Total: n multiplications + n additions.

Time complexity: $O(n)$, Space complexity: $O(1)$.

Proof of Correctness: We use a loop invariant. At the start of iteration i , let p hold the value of the partial polynomial:

$$p = a_{i+1} + x_0(a_{i+2} + x_0(a_{i+3} + \dots + x_0(a_n))).$$

When the loop finishes ($i = 0$), $p = P(x_0)$.

Thus, Horner's method correctly evaluates $P(x_0)$.

Comparison

Method	Multiplications	Time Complexity
Naïve Evaluation	$\frac{n(n+1)}{2}$	$O(n^2)$
Horner's Rule (With Preprocessing)	n	$O(n)$

Remarks

- Horner's method is numerically more stable and efficient.
- It is the standard technique used in polynomial interpolation, curve fitting, and compiler code generation.
- Preprocessing refers to rewriting the polynomial into nested form before evaluation.

Fast Matrix Multiplication Algorithms: Winograd and Strassen

Problem Statement

Given two square matrices

$$A = [a_{ij}]_{n \times n}, \quad B = [b_{ij}]_{n \times n},$$

compute their product

$$C = A \times B, \quad \text{where } c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}.$$

The conventional algorithm takes $O(n^3)$ time using triple nested loops.

Standard Matrix Multiplication

Complexity:

$$\text{Multiplications: } n^3, \quad \text{Additions: } n^3 - n^2, \quad \text{Time: } O(n^3).$$

—

Algorithm 4 Standard_Matrix_Multiply(A, B, n)

Require: Two $n \times n$ matrices A and B

Ensure: Product matrix $C = A \times B$

```
1: for  $i = 1$  to  $n$  do
2:   for  $j = 1$  to  $n$  do
3:      $C[i][j] \leftarrow 0$ 
4:     for  $k = 1$  to  $n$  do
5:        $C[i][j] \leftarrow C[i][j] + A[i][k] \times B[k][j]$ 
6:     end for
7:   end for
8: end for
9: return  $C$ 
```

Winograd's Algorithm (Improved Classical Method)

Key Idea: Winograd observed that in computing $c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}$, partial products can be reused by preprocessing row and column factors.

Preprocessing:

$$\begin{aligned} \text{rowFactor}[i] &= \sum_{k=1}^{\lfloor n/2 \rfloor} a_{i,2k-1} \times a_{i,2k}, \\ \text{colFactor}[j] &= \sum_{k=1}^{\lfloor n/2 \rfloor} b_{2k-1,j} \times b_{2k,j}. \end{aligned}$$

Computation: Then for each i, j :

$$c_{ij} = -\text{rowFactor}[i] - \text{colFactor}[j] + \sum_{k=1}^{\lfloor n/2 \rfloor} (a_{i,2k-1} + b_{2k,j})(a_{i,2k} + b_{2k-1,j}).$$

If n is odd, add the remaining term:

$$c_{ij} + = a_{i,n} \times b_{n,j}.$$

Complexity: Winograd's method reduces the number of multiplications by almost 25% compared to the naïve algorithm:

$$\text{Multiplications: } \approx \frac{2}{3}n^3, \quad \text{Time: } O(n^3).$$

Although still cubic, it performs fewer arithmetic operations and improves cache utilization.

Algorithm 5 Winograd_Matrix_Multiply(A, B, n)

Require: A, B : $n \times n$ matrices

Ensure: $C = A \times B$

```
1: for  $i = 1$  to  $n$  do
2:    $rowFactor[i] \leftarrow 0$ 
3:   for  $k = 1$  to  $\lfloor n/2 \rfloor$  do
4:      $rowFactor[i] \leftarrow rowFactor[i] + A[i][2k-1] \times A[i][2k]$ 
5:   end for
6: end for
7: for  $j = 1$  to  $n$  do
8:    $colFactor[j] \leftarrow 0$ 
9:   for  $k = 1$  to  $\lfloor n/2 \rfloor$  do
10:     $colFactor[j] \leftarrow colFactor[j] + B[2k-1][j] \times B[2k][j]$ 
11:  end for
12: end for
13: for  $i = 1$  to  $n$  do
14:   for  $j = 1$  to  $n$  do
15:     $C[i][j] \leftarrow -rowFactor[i] - colFactor[j]$ 
16:    for  $k = 1$  to  $\lfloor n/2 \rfloor$  do
17:       $C[i][j] \leftarrow C[i][j] + (A[i][2k-1] + B[2k][j]) \times (A[i][2k] + B[2k-1][j])$ 
18:    end for
19:    if  $n$  is odd then
20:       $C[i][j] \leftarrow C[i][j] + A[i][n] \times B[n][j]$ 
21:    end if
22:  end for
23: end for
24: return  $C$ 
```

Strassen's Algorithm (Divide-and-Conquer Method)

Key Idea: Strassen (1969) showed that two 2×2 matrix multiplications can be performed using only 7 multiplications and 18 additions, instead of 8 multiplications.

Matrix Partition: For even n , split

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}.$$

Compute:

$$\begin{aligned}M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), \\M_2 &= (A_{21} + A_{22})B_{11}, \\M_3 &= A_{11}(B_{12} - B_{22}), \\M_4 &= A_{22}(B_{21} - B_{11}), \\M_5 &= (A_{11} + A_{12})B_{22}, \\M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}).\end{aligned}$$

Then combine:

$$\begin{aligned}C_{11} &= M_1 + M_4 - M_5 + M_7, \\C_{12} &= M_3 + M_5, \\C_{21} &= M_2 + M_4, \\C_{22} &= M_1 - M_2 + M_3 + M_6.\end{aligned}$$

Algorithm 6 Strassen_Matrix_Multiply(A, B, n)

Require: Two $n \times n$ matrices A and B

Ensure: Product matrix $C = A \times B$

```
1: if  $n = 1$  then
2:   return  $[A[1][1] \times B[1][1]]$ 
3: else
4:   Partition  $A$  and  $B$  into submatrices of size  $(n/2) \times (n/2)$ 
5:   Compute  $M_1$  through  $M_7$  as above
6:   Combine  $M_1$ – $M_7$  to form  $C_{11}, C_{12}, C_{21}, C_{22}$ 
7:   Merge to form  $C$ 
8:   return  $C$ 
9: end if
```

Proof Sketch of Correctness: Each M_i captures a linear combination of A_{pq} and B_{rs} terms, chosen so that the recombination step yields the exact same result as conventional multiplication. The correctness can be verified by algebraic expansion of the four resulting submatrices.

Complexity: Let $T(n)$ be the running time.

$$T(n) = 7T(n/2) + O(n^2)$$

Solving by the Master Theorem gives:

$$T(n) = O(n^{\log_2 7}) \approx O(n^{2.8074}).$$

Thus, Strassen's algorithm is asymptotically faster than the classical $O(n^3)$ method.

Space Complexity: Additional $O(n^2)$ space is required for temporary matrices.

Comparison of Algorithms

Algorithm	Multiplications	Asymptotic Time	Remarks
Naïve	n^3	$O(n^3)$	Simple triple loop
Winograd	$\approx \frac{2}{3}n^3$	$O(n^3)$	Reduces constant factors
Strassen	$n^{\log_2 7} \approx n^{2.81}$	$O(n^{2.81})$	Divide-and-conquer; faster asymptotically

Remarks

- Winograd’s method is an arithmetic optimization of the standard algorithm — beneficial for small matrices.
- Strassen’s algorithm is asymptotically faster but incurs higher constant factors and numerical instability; best for large matrices or symbolic computation.
- Further improvements (e.g., Coppersmith–Winograd, 1987) reduced the exponent to ≈ 2.376 .

1 Applications of Winograd’s and Strassen’s Algorithms

1.1 Applications of Winograd’s Algorithm

Winograd’s algorithm is a refinement of the standard matrix multiplication method that minimizes the number of arithmetic operations, especially multiplications. It is particularly effective when dealing with large matrices or when multiplications are more costly than additions.

- **Signal and Image Processing:** Employed in convolution operations, filtering, and image transformations where large-scale matrix multiplications occur.
- **Scientific Computing:** Used in large-scale linear algebra computations such as solving systems of equations or eigenvalue problems where efficiency is critical.
- **Embedded and Low-Power Systems:** Suitable for DSPs and neural accelerators where minimizing multiplications reduces power consumption.
- **Deep Learning:** The Winograd minimal filtering algorithm is widely used to accelerate convolution layers in modern convolutional neural networks (CNNs).

1.2 Applications of Strassen's Algorithm

Strassen's algorithm applies a divide-and-conquer strategy that reduces the asymptotic time complexity of matrix multiplication from $O(n^3)$ to approximately $O(n^{2.81})$. It is most effective for large, dense matrices.

- **Computational Linear Algebra:** Applied to matrix inversion, determinant computation, and solving linear systems involving high-dimensional data.
- **Computer Graphics and Simulations:** Utilized for geometric transformations and complex physical simulations requiring heavy matrix operations.
- **Scientific and Engineering Computations:** Helps in large-scale simulations such as finite element methods and molecular dynamics.
- **High-Performance Computing (HPC):** Integrated into optimized BLAS libraries and HPC packages for fast dense matrix operations.
- **Theoretical Computer Science:** Serves as a foundation for more advanced fast matrix multiplication algorithms (e.g., Coppersmith–Winograd and later improvements).

1.3 Comparison of Applications

Aspect	Winograd's Algorithm	Strassen's Algorithm
Optimization focus	Reduction in number of multiplications using arithmetic rearrangements	Reduction in asymptotic time complexity via divide-and-conquer
Efficiency for small matrices	High	Moderate
Efficiency for large matrices	Moderate	High
Numerical stability	More stable due to fewer recursive subtractions	Less stable due to recursive subtractions and rounding errors
Practical usage	Deep learning frameworks, DSPs, CNN convolutions	HPC systems, dense matrix libraries, theoretical research
Implementation complexity	Low to medium	High

Table 1: Comparison of Applications of Winograd's and Strassen's Algorithms

1.4 Limitations and Trade-offs

- **Numerical Stability:** Strassen's algorithm can suffer from numerical instability due to repeated subtraction of intermediate matrices, leading to rounding errors.

Winograd's variant is generally more stable.

- **Memory Overhead:** Strassen's recursive nature increases memory requirements, which may offset performance gains for moderately sized matrices.
- **Implementation Complexity:** Strassen's method requires careful implementation to avoid overhead from recursion and temporary storage. Winograd's algorithm is simpler and easier to integrate into standard matrix multiplication frameworks.
- **Threshold Size:** Strassen's algorithm is beneficial only beyond a certain matrix size (typically $n > 128$ or $n > 256$), below which conventional or Winograd multiplication is faster.
- **Parallelization:** While Strassen's algorithm can be parallelized, managing recursion efficiently on parallel architectures is complex. Winograd's approach offers more predictable performance in parallel and embedded systems.
- **Practical Preference:** Modern applications such as CNNs often prefer Winograd's algorithm for small convolution kernels, whereas Strassen's algorithm finds use in scientific or theoretical contexts requiring large matrix multiplications.

2 Fast Fourier Transform (FFT)

2.1 Motivation: Polynomial Multiplication

Consider two polynomials of degree at most $n - 1$:

$$A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

$$B(x) = b_0 + b_1x + b_2x^2 + \cdots + b_{n-1}x^{n-1}$$

Their product is:

$$C(x) = A(x) \cdot B(x) = \sum_{k=0}^{2n-2} c_k x^k$$

where

$$c_k = \sum_{i=0}^k a_i b_{k-i}$$

Naive Approach: Direct computation of each c_k requires $O(n^2)$ operations, which becomes inefficient for large n .

2.2 Key Idea: Using Evaluation and Interpolation

Instead of multiplying coefficients directly, we can:

1. Evaluate both $A(x)$ and $B(x)$ at sufficiently many points.
2. Multiply the values pointwise.
3. Reconstruct (interpolate) the product polynomial from these points.

By the fundamental theorem of algebra, a degree $(2n - 2)$ polynomial is uniquely determined by its values at $2n - 1$ distinct points.

2.3 Evaluation at Roots of Unity

To make evaluation and interpolation efficient, we choose the n -th roots of unity:

$$\omega_n = e^{2\pi i/n}$$

$$1, \omega_n, \omega_n^2, \dots, \omega_n^{n-1}$$

These points have special symmetry properties that allow us to compute evaluations recursively.

2.4 Discrete Fourier Transform (DFT)

The DFT of a sequence $a_0, a_1, \dots, a_{n-1}$ is defined as:

$$A_k = \sum_{j=0}^{n-1} a_j \omega_n^{jk}, \quad 0 \leq k < n$$

The inverse DFT is:

$$a_j = \frac{1}{n} \sum_{k=0}^{n-1} A_k \omega_n^{-jk}, \quad 0 \leq j < n$$

Naively, both DFT and its inverse take $O(n^2)$ time.

2.5 Fast Fourier Transform (FFT)

The FFT algorithm computes the DFT in $O(n \log n)$ time using a divide-and-conquer strategy.

2.5.1 Core Idea

Split the polynomial into even and odd parts:

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2)$$

Then the DFT can be computed recursively:

$$A(\omega_n^k) = A_{\text{even}}(\omega_{n/2}^k) + \omega_n^k A_{\text{odd}}(\omega_{n/2}^k)$$

$$A(\omega_n^{k+n/2}) = A_{\text{even}}(\omega_{n/2}^k) - \omega_n^k A_{\text{odd}}(\omega_{n/2}^k)$$

Thus, we compute two FFTs of size $n/2$ and combine results in $O(n)$ time.

2.6 FFT Algorithm (Recursive Form)

Algorithm 7 FFT Algorithm

```

1: procedure FFT( $a, n$ )
2:   if  $n = 1$  then
3:     return  $a$ 
4:   end if
5:   Let  $\omega_n = e^{2\pi i/n}$ 
6:   Split  $a$  into even and odd indexed coefficients:  $a_{\text{even}}, a_{\text{odd}}$ 
7:    $y_{\text{even}} = \text{FFT}(a_{\text{even}}, n/2)$ 
8:    $y_{\text{odd}} = \text{FFT}(a_{\text{odd}}, n/2)$ 
9:   for  $k = 0$  to  $n/2 - 1$  do
10:     $t = \omega_n^k \cdot y_{\text{odd}}[k]$ 
11:     $y[k] = y_{\text{even}}[k] + t$ 
12:     $y[k + n/2] = y_{\text{even}}[k] - t$ 
13:   end for
14:   return  $y$ 
15: end procedure

```

2.7 Inverse FFT

The Inverse FFT (IFFT) follows the same structure as FFT, but uses ω_n^{-1} in place of ω_n and divides the final result by n .

2.8 Proof of Correctness

The correctness of FFT follows from the recursive decomposition of the DFT formula:

$$\begin{aligned}
A_k &= \sum_{j=0}^{n-1} a_j \omega_n^{jk} = \sum_{j=0}^{n/2-1} a_{2j} \omega_n^{2jk} + \omega_n^k \sum_{j=0}^{n/2-1} a_{2j+1} \omega_n^{2jk} \\
&= A_{\text{even}}(\omega_{n/2}^k) + \omega_n^k A_{\text{odd}}(\omega_{n/2}^k)
\end{aligned}$$

Since this matches the definition of DFT on the subproblems, the recursion correctly computes all A_k . The same logic holds for the inverse transform.

2.9 Complexity Analysis

Let $T(n)$ denote the time complexity of FFT.

$$T(n) = 2T(n/2) + O(n)$$

By the Master Theorem:

$$T(n) = O(n \log n)$$

Comparison:

Method	Time Complexity
Naive Polynomial Multiplication	$O(n^2)$
FFT-based Polynomial Multiplication	$O(n \log n)$

Table 2: Comparison of Polynomial Multiplication Methods

2.10 Applications of FFT

- **Polynomial Multiplication:** Fast multiplication of large-degree polynomials using DFT-based convolution.
- **Signal Processing:** Computation of frequency components in digital signals (Discrete Fourier Transform).
- **Image Processing:** Used for 2D FFTs in image compression, filtering, and enhancement.
- **Numerical Methods:** Solving partial differential equations using spectral methods.
- **Audio Compression and Speech Analysis:** FFT is the foundation of MP3 and speech recognition algorithms.
- **Cryptography and Large Integer Arithmetic:** FFT-based multiplication (e.g., Schönhage–Strassen algorithm) is used in RSA key generation and cryptanalysis.
- **Pattern Matching and Convolution:** Efficient computation of correlations and convolution in $O(n \log n)$.

2.11 Importance of FFT

The FFT is one of the most influential algorithms in computational mathematics and engineering due to:

- Its dramatic reduction in computational complexity from $O(n^2)$ to $O(n \log n)$.
- Its applicability across almost every domain involving signals, images, or large numeric data.
- Its role in enabling real-time digital processing and scientific computation at scale.

3 Important Applications of the FFT

The Fast Fourier Transform (FFT) is a highly efficient algorithm to compute the Discrete Fourier Transform (DFT) and its inverse. Its speed and versatility have made it indispensable across scientific, engineering, and mathematical disciplines. Below are important applications with relevant details:

1. **Signal Processing and Spectrum Analysis:** FFT is widely used to convert time-domain signals into their frequency-domain representations. This enables the identification of frequency components, filtering of signals, noise reduction, and the analysis of vibration patterns or harmonic content in mechanical systems [?, ?, ?].
2. **Audio and Speech Analysis:** In audio engineering, FFT facilitates tasks such as pitch detection, sound compression, echo removal, and equalization by transforming audio signals and extracting their frequency content [?, ?].
3. **Image Processing:** FFT algorithms are used for image filtering, image compression (JPEG, MPEG), and pattern recognition by converting spatial data to frequency domain and simplifying convolution operations [?, ?, ?].
4. **Wireless Communication and OFDM:** FFT enables efficient modulation and demodulation in digital communication systems, especially in Orthogonal Frequency Division Multiplexing (OFDM) for Wi-Fi, 4G/5G, and DSL by converting signals into orthogonal subcarriers [?, ?].
5. **Radar and Seismic Data Analysis:** In radar systems, FFT is used for pulse compression and target detection. Similarly, FFT helps analyze seismic signals, improving the detection and characterization of geophysical phenomena [?, ?].
6. **Time Series Analysis:** FFT is valuable for identifying repeating patterns or periodic signals in large data sets, such as climate data, stock trends, and physiological measurements – efficiently revealing the underlying cyclical phenomena [?].
7. **Scientific Computation:** FFT is essential for solving partial differential equations, analyzing crystallography data (X-ray diffraction, NMR), and big data processing in astronomy and physics [?, ?].

3.1 Role of FFT in Signal Processing and the Shift from Analog to Digital

The Fast Fourier Transform (FFT) is a pivotal tool in digital signal processing because it efficiently converts a discrete time-domain signal into its frequency-domain representation [?]. This frequency analysis is fundamental for understanding the underlying components of a signal, such as detecting periodicities, harmonics, or sources of noise.

In particular, FFT replaced or augmented many traditional analog signal analysis techniques. In analog systems, frequency analysis required components such as tuned filters, oscilloscopes, and spectrum analyzers—all limited in speed, flexibility, accuracy, and scalability. FFT, on the other hand, offers the following:

- **High Computational Efficiency:** The classic Discrete Fourier Transform (DFT) demands $\mathcal{O}(N^2)$ operations for N points. FFT optimizes this to $\mathcal{O}(N \log N)$, allowing for large-scale and real-time analysis.
- **Wide Applicability:** FFT can be implemented in software, firmware, or digital hardware, making it ideal for integration with digital recording, storage, and communication systems.
- **Precision and Flexibility:** Digital signals can be sampled, stored, filtered, and analyzed repeatedly without loss, providing much higher fidelity than analog circuits for most applications.

The shift to digital signal processing (DSP) was triggered by the availability and power of the FFT algorithm, which allowed for:

1. *Real-time spectrum analysis:* Engineers can visualize the spectral content of signals instantly, diagnose problems, and design digital filters with unprecedented flexibility.
2. *Noise reduction and feature extraction:* FFT enables the development of adaptive digital filters that can remove unwanted noise and extract relevant information from complex signals.
3. *Compression and transmission:* Techniques such as MP3, JPEG, and OFDM (used in Wi-Fi, LTE) rely on FFT for encoding, compression, and efficient multi-channel signal transmission.
4. *Seamless integration with digital devices:* FFT has allowed one to analyze and modify recorded audio, video, and sensor data on computers and embedded processors in ways that are impossible with analog hardware alone.

In summary, FFT made working in the frequency domain as practical as in the time domain, thus transforming the landscape of signal processing from hardware-centric analog methods to flexible, powerful, and scalable digital approaches [?, ?].

4 Knuth–Morris–Pratt (KMP) String Matching Algorithm

Problem Statement

Given a text string $T[0 \dots n - 1]$ and a pattern string $P[0 \dots m - 1]$, find all occurrences of P in T (i.e., all indices i such that $T[i \dots i + m - 1] = P[0 \dots m - 1]$). We assume characters come from some finite alphabet; n and m denote the lengths of T and P respectively.

Key Idea

Naïve pattern matching may backtrack in the text, causing $O(nm)$ worst-case time. KMP avoids reexamining characters of the text by preprocessing the pattern to compute the longest proper prefix that is also a suffix (the “prefix-function” or **lps** / failure function). During the search, when a mismatch occurs after k matched characters, the algorithm uses **lps**[$k-1$] to shift the pattern by the maximal safe amount — ensuring that each text character is examined at most once. This yields an overall running time of $O(n + m)$.

Prefix Function (LPS) — Definitions

For pattern $P[0 \dots m - 1]$, define the prefix-function (also called **lps**):

$$\text{lps}[q] = \max\{ \ell \mid 0 \leq \ell < q + 1, P[0 \dots \ell - 1] = P[q - \ell + 1 \dots q] \}.$$

In words, **lps**[q] is the length of the longest proper prefix of $P[0 \dots q]$ that is also a suffix of $P[0 \dots q]$.

Pseudocode

Algorithm 8 Compute_LPS(P, m)

Require: Pattern $P[0 \dots m - 1]$

Ensure: Array $\text{lps}[0 \dots m - 1]$

```

1:  $\text{lps}[0] \leftarrow 0$ 
2:  $\text{len} \leftarrow 0$  ▷ length of previous longest prefix-suffix
3:  $i \leftarrow 1$ 
4: while  $i < m$  do
5:   if  $P[i] = P[\text{len}]$  then
6:      $\text{len} \leftarrow \text{len} + 1$ 
7:      $\text{lps}[i] \leftarrow \text{len}$ 
8:      $i \leftarrow i + 1$ 
9:   else
10:    if  $\text{len} \neq 0$  then
11:       $\text{len} \leftarrow \text{lps}[\text{len} - 1]$  ▷ fall back ▷ do not increment  $i$  here
12:    else
13:       $\text{lps}[i] \leftarrow 0$ 
14:       $i \leftarrow i + 1$ 
15:    end if
16:  end if
17: end while
18: return  $\text{lps}$ 

```

Algorithm 9 KMP_Search(T, n, P, m)

Require: Text $T[0 \dots n - 1]$, Pattern $P[0 \dots m - 1]$

Ensure: All start indices s where P occurs in T

```

1:  $\text{lps} \leftarrow \text{Compute\_LPS}(P, m)$ 
2:  $i \leftarrow 0$  ▷ index in  $T$ 
3:  $j \leftarrow 0$  ▷ index in  $P$ 
4: while  $i < n$  do
5:   if  $T[i] = P[j]$  then
6:      $i \leftarrow i + 1$ 
7:      $j \leftarrow j + 1$ 
8:     if  $j = m$  then
9:       report occurrence at index  $i - m$ 
10:       $j \leftarrow \text{lps}[j - 1]$  ▷ continue searching for next match
11:    end if
12:  else
13:    if  $j \neq 0$  then
14:       $j \leftarrow \text{lps}[j - 1]$  ▷ use failure function; do not increment  $i$ 
15:    else
16:       $i \leftarrow i + 1$ 
17:    end if
18:  end if
19: end while

```

Proof of Correctness

We give correctness in two parts: (A) correctness of the prefix-function computation, and (B) correctness of the search using `lps`.

(A) Correctness of `Compute_LPS`. We maintain the invariant that at the start of each iteration with indices (i, len) :

`lps` $[0 \dots i-1]$ are correctly computed, and len = length of longest prefix-suffix for $P[0 \dots i-1]$.

Base: `lps` $[0] = 0$ and $len = 0$ hold. Inductive step: when $P[i] = P[len]$, we extend the known longest prefix-suffix by one, so `lps` $[i] = len + 1$ is correct. When $P[i] \neq P[len]$ and $len \neq 0$, we set $len \leftarrow \text{lps}[len - 1]$, which by induction is the next candidate for the longest prefix-suffix; repeating this fallback eventually finds the correct maximal ℓ (possibly 0). Thus the algorithm computes the intended longest proper prefix-suffix lengths for each position.

(B) Correctness of `KMP_Search`. We prove that during the search the pair (i, j) always satisfies:

$$T[i - j \dots i - 1] = P[0 \dots j - 1]$$

i.e., the last j characters of the prefix $T[0 \dots i - 1]$ match the first j pattern characters. This is true initially ($i = j = 0$). When $T[i] = P[j]$ we increment both indices, preserving the invariant. On mismatch, if $j \neq 0$ we set $j \leftarrow \text{lps}[j - 1]$. By definition of `lps`, the new j is the length of the longest prefix of P that is a proper suffix of $P[0 \dots j - 1]$; hence after fallback the invariant holds without advancing i (we have reduced the matched prefix of P but the matched suffix in T remains aligned). If $j = 0$, no prefix of P matches the current position, so we must advance i .

Thus whenever j reaches m , $P[0 \dots m - 1]$ matches $T[i - m \dots i - 1]$ and reporting $i - m$ is correct. The algorithm never overlooks a potential match because all possible alignments are implicitly considered via the `lps`-guided fallbacks. Therefore KMP reports exactly the occurrences of P in T .

Complexity Analysis

`Compute_LPS`: Each iteration either increments i or reduces len (by jumping to `lps` $[len-1]$). Since len only decreases at most m times and i increases up to m , the total work is $O(m)$. Thus `Compute_LPS` runs in $O(m)$ time and $O(m)$ space (for the `lps` array).

`KMP_Search`: Similarly, in the search phase each step either increments i or reduces j via `lps` $[j-1]$. Each of i and j is bounded by n and m respectively and decreases/increases

a bounded number of times. Therefore the search takes $O(n)$ time after preprocessing. Total time is $O(n + m)$ and extra space is $O(m)$.

Example

Pattern $P = \text{A B A B A C}$ (indices $0 \dots 5$), $m = 6$. Compute lps :

index i	0	1	2	3	4	5
$P[i]$	A	B	A	B	A	C
$\text{lps}[i]$	0	0	1	2	3	0

Explanation: prefixes/suffixes:

- $\text{lps}[2] = 1$ since $P[0] = P[2] = \text{A}$.
- $\text{lps}[4] = 3$ since $P[0 \dots 2] = \text{ABA}$ equals suffix $P[2 \dots 4] = \text{ABA}$.
- $\text{lps}[5] = 0$ because C breaks the border.

Now let text $T = \text{A B A B A B A C}$ (indices $0 \dots 7$). Run KMP:

- Match proceeds: j grows A,B,A,B,A to $j = 5$ at $i = 5$ (characters matched).
- At $i = 5$, compare $T[5] = B$ with $P[5] = C$: mismatch. Fallback $j \leftarrow \text{lps}[4] = 3$.
- Now compare $T[5] = B$ with $P[3] = B$: match $\Rightarrow i \leftarrow 6, j \leftarrow 4$; continue.
- Continue and eventually at $i = 8$ (end) when $j = 6$ we find occurrence at index $i - m = 2$ corresponding to substring $T[2 \dots 7] = \text{ABABAC}$.

Thus KMP finds the occurrence starting at index 2 and does so without re-scanning characters unnecessarily.

Remarks

- KMP is stable and deterministic, suitable for worst-case linear-time pattern matching.
- The prefix-function (lps) may be generalized for other pattern matching variants (e.g., algorithms for automaton construction).
- Practical implementations often store lps and then use the described search loop; careful indexing (0-based vs 1-based) must be consistent.
- KMP is particularly useful when matching the same pattern against many texts (one-time preprocessing cost) or when worst-case guarantees matter (e.g., in adversarial inputs).